

Codex_Step_by_Step_Hands_On_Training

Codex 从零到深入：逐步动手培训手册（本地文件夹版）

场景：Windows + VS Code + Java + Maven

项目：Task Tracker（命令行待办事项程序）

版本：2026-06-23，本地文件夹版

适合：会一点 Java、想把 Codex 作为可靠开发搭档的学习者。

本版的重要说明：不使用 Git，也不使用 GitHub

这份教程**不要求安装 Git，不需要 GitHub 账号，也不会出现提交、分支、远程仓库、PR、worktree 等步骤。**

为了降低学习复杂度，我们用下面的方式保护代码：

1. 每个大步骤开始前，创建一个**本地快照文件夹**。
2. 每次修改后，自己运行测试并查看 Codex 的改动说明。
3. 想比较修改前后时，用 VS Code 的文件比较功能打开“快照文件”和“当前文件”。
4. 出现问题时，不覆盖当前代码；而是把某个快照复制为一个新的“恢复版本”文件夹，再打开它。

这套方法适合你在学习 Codex 和 Java 项目流程。以后真正做课程项目、团队项目或实习项目时，再单独学习 Git/GitHub 即可。

目录

1. 最终成果与学习路线
2. 开始前的环境检查
3. 第一次配置 VS Code 中的 Codex
4. 实验 0：创建本地项目文件夹与快照机制
5. 实验 1：只读分析与精确提供上下文
6. 实验 2：从需求到可运行 Java 项目
7. 实验 3：读懂 Codex 生成的项目
8. 实验 4：新增小功能，练习测试优先

9. 实验 5：故意制造 bug，再让 Codex 用证据修复
 10. 实验 6：多文件功能与 Plan mode
 11. 实验 7：重构而不改变行为
 12. 实验 8：本地代码审查与文件比较
 13. 实验 9：补 README 与交付说明
 14. 实验 10：用 AGENTS.md 固化项目规则
 15. 实验 11：Codex CLI：交互、一次性提问、自动化输出
 16. 实验 12：Codex App：并行线程与只读自动化
 17. 实验 13：创建个人 Skill
 18. 实验 14：MCP 的安全入门
 19. 迁移到你的真实 Java 项目
 20. 常用提示词模板
 21. 无 Git 的恢复与排错清单
 22. 最终验收清单
 23. 官方参考资料
-

1. 最终成果与学习路线

1.1 你完成本教程后，能独立做什么

你会完整练一遍这些高频能力：

- 在 VS Code 中安装、登录并安全使用 Codex。
- 让 Codex **只读分析**陌生代码，而不是一开始就乱改。
- 使用 Plan mode，先把需求拆成实施计划，再决定是否让它写代码。
- 从空文件夹生成一个 Java + Maven 项目，并亲自验证构建与测试。
- 通过 @文件、选中代码、终端报错等方式给 Codex 精准上下文。
- 让 Codex 用测试优先的方式新增功能、修复 bug、重构代码、补文档。
- 用“本地快照 + 文件比较”审查改动，不依赖任何代码管理平台。
- 用 AGENTS.md 保存项目规则，减少重复提示。
- 使用 Codex CLI 做只读分析、交互式会话、一次性任务和报告输出。
- 在 Codex App 里练习项目线程、审查结果和低风险的只读自动化。

- 创建个人 Skill，并理解 MCP 什么时候值得使用。

本教程原则：Codex 可以读、写、运行，但“是否正确”和“是否保留改动”始终由你负责。每次大改前创建本地快照；每次修改后跑测试；任何你不理解的命令先不要批准。

1.2 建议学习节奏

阶段	实验	重点	结束时应留下的成果
A. 上手	0-3	环境、登录、只读分析、生成项目	可运行的 Java Maven 项目
B. 日常开发	4-6	小功能、测试、调试、计划	多个通过的 JUnit 测试
C. 质量控制	7-10	重构、review、文档、规则	README、AGENTS.md、项目快照
D. 进阶工作流	11-14	CLI、App、Skills、MCP	个人 Skill、只读自动化报告

建议分 4 次完成，每次 1.5-2 小时。不要一次做完后半段；真正理解 Codex 的关键是：**每个实验都自己运行命令并看结果。**

1.3 最终项目约定

整份教程固定使用以下约定。若 Codex 生成的名称不同，先让它整理一致再继续。

- Maven 坐标：`com.janel.codexlab:task-tracker`
- Java package：`com.janel.codexlab`
- Java 版本：17
- 主类：`com.janel.codexlab.App`
- 核心类：`Task`、`TaskService`
- 测试框架：JUnit 5
- 测试命令：`mvn test`
- 打包命令：`mvn package`
- 运行命令：`java -cp target/classes com.janel.codexlab.App`

2. 开始前的环境检查

2.1 需要准备什么

最低需要：

1. Windows 电脑。
2. VS Code。
3. JDK 17 或更高版本。
4. Maven。
5. ChatGPT 账号。建议使用你平时的 ChatGPT 账号登录 Codex。

本教程**不需要 Git，也不需要 GitHub**。

Codex 的 IDE 扩展、CLI 与 Windows App 都支持 ChatGPT 登录；API key 登录走 API 平台的独立计费路径。练习时优先使用 ChatGPT 登录，不要把 API key 粘贴到聊天、源代码或配置文件里。

2.2 在 VS Code 中检查 Java 与 Maven

第 1 步：打开 VS Code 终端

1. 打开 VS Code。
2. 点击顶部菜单 **Terminal**（终端）。
3. 点击 **New Terminal**（新建终端）。
4. 确认终端标签中显示的是 **PowerShell**。

快捷键是：

代码块

```
1  Ctrl + `
```

第 2 步：逐条运行检查命令

在 PowerShell 中逐条输入，输入一条就按一次 Enter：

代码块

```
1  java -version
2  javac -version
3  mvn -v
```

你应该看到什么

- `java -version`：Java 17 或更高版本。

- `javac -version`：Java 编译器版本。
- `mvn -v`：Maven 版本、Java 版本与 Java home 路径。

常见错误

看到的提示	常见原因	先做什么
<code>java is not recognized</code>	JDK 未安装或 PATH 未刷新	安装 JDK 17+，完全关闭再重新打开 VS Code
只有 <code>java</code> ，没有 <code>javac</code>	安装的是 JRE，不是完整 JDK	安装 JDK
<code>mvn is not recognized</code>	Maven 未安装或 PATH 未刷新	安装 Maven 后重开 VS Code
Maven 使用 Java 8/11	环境变量指向旧 JDK	修改 <code>JAVA_HOME</code> 后重开 VS Code

停在这里直到三条命令都有正常结果。后面的构建、测试和 CLI 练习都依赖它们。

2.3 安全边界：第一次使用保持可控

Codex 可以读取文件、修改文件、运行命令。对新手来说，建议默认遵守下面的边界：

1. 只让 Codex 操作 `task-tracker` 这个练习文件夹。
2. 第一次使用时选择默认的安全权限或普通 Agent，不选择 Full Access。
3. 看到安装依赖、访问网络、写入项目外目录的请求，先看命令再批准。
4. 看到会删除大量文件、修改系统目录、读取凭据的命令，先拒绝并问原因。
5. 每个功能完成后，必须自己执行一次 `mvn test`。

通常合理的命令

- `mvn test`
- `mvn package`
- `Get-ChildItem`
- 在当前项目中创建 `src`、`README.md`、测试文件

需要特别谨慎的命令

- `Remove-Item -Recurse -Force`
- 下载并执行不明脚本
- 要求读取 `.ssh`、浏览器凭据、密码管理器、`auth.json`、API key
- 写入桌面、文档目录以外的陌生位置

3. 第一次配置 VS Code 中的 Codex

本教程以 **VS Code 扩展** 为主要入口；CLI 和 App 放到后面再学。

3.1 安装 Java 开发扩展

第 1 步：打开 Extensions

按：

代码块

```
1  Ctrl + Shift + X
```

或者点击左侧像四个小方块的 **Extensions（扩展）** 图标。

第 2 步：搜索并安装

在搜索框输入：

代码块

```
1  Extension Pack for Java
```

选择可信发布者提供的 Java 扩展包，点击 **Install（安装）**。

第 3 步：重新加载窗口

若 VS Code 提示 **Reload**，点击它。

3.2 安装 Codex 扩展

第 1 步：继续留在 Extensions 页面

搜索：

代码块

```
1  Codex
```

第 2 步：确认发布者

选择由 **OpenAI** 发布的 Codex 扩展。不要安装名称很像但发布者不明的扩展。

第 3 步：打开 Codex 面板

安装完成后：

1. 点击 **Reload**，或关闭后重新打开 VS Code。
2. 在侧边栏寻找 Codex 图标。
3. 看不到时按 `Ctrl + Shift + P`。
4. 输入：

代码块

```
1 Codex: Open Sidebar
```

5. 按 Enter。

3.3 使用 ChatGPT 账号登录

首次打开 Codex 面板，选择：

代码块

```
1 Sign in with ChatGPT
```

浏览器会打开登录页。登录你平时用 ChatGPT 的账号，完成授权后回到 VS Code。

不作为本教程默认方式的选项

代码块

```
1 Sign in with API key
```

API key 不是 Plus 订阅的“密码替代品”。它通常走独立的 API 使用和计费路径，也更容易因为复制、保存不当而泄露。

登录后做一个无风险测试

在 Codex 对话框输入：

代码块

- 1 请只回答：你现在可以在 VS Code 的项目文件夹中读取文件、提出修改建议和运行命令，但在没有我确认前不要修改任何文件。不要执行命令。

通过标准： Codex 用自然语言回应，不创建文件、不运行命令。

4. 实验 0：创建本地项目文件夹与快照机制

目标： 创建一个只用来学习的项目目录，并学会在大修改前做本地快照。

4.1 创建训练根目录

第 1 步：在 VS Code PowerShell 中输入

代码块

```
1 $root = Join-Path $HOME "Documents\CodexTraining"
2 $project = Join-Path $root "task-tracker"
3 $backupRoot = Join-Path $root "snapshots"
4 New-Item -ItemType Directory -Force -Path $project, $backupRoot | Out-Null
5 code $project
```

第 2 步：在新窗口中确认文件夹

打开 VS Code 后，左侧 Explorer 应显示一个空的 `task-tracker` 文件夹。

如果 VS Code 弹出“是否信任此文件夹”，选择你自己的本地练习目录后，点击 **Trust**。

4.2 创建一个 `notes.md`

在 Explorer 中：

1. 点击 `task-tracker` 右侧的新建文件图标。
2. 输入文件名：

代码块

```
1 notes.md
```

3. 写入：

代码块

```
1 # Codex Training Notes
```


- 2
- 3 - 项目: Task Tracker
- 4 - 目标: 学习如何安全地使用 Codex
- 5 - 规则: 每次大改前先建立本地快照, 改完后运行 `mvn test`。

4. 按 `Ctrl + S` 保存。

4.3 创建本地快照函数

这个函数只会复制你的练习项目到 `Documents\CodexTraining\snapshots`, 不会上传任何内容。

第 1 步: 打开一个新的 PowerShell 终端

在 VS Code 中选择 **Terminal** → **New Terminal**。

第 2 步: 完整粘贴下面代码

代码块

```
1 function New-TrainingSnapshot {
2     param(
3         [Parameter(Mandatory = $true)]
4         [string]$Label
5     )
6
7     $root = Join-Path $HOME "Documents\CodexTraining"
8     $project = Join-Path $root "task-tracker"
9     $backupRoot = Join-Path $root "snapshots"
10    $destination = Join-Path $backupRoot $Label
11
12    if (Test-Path $destination) {
13        Write-Error "同名快照已经存在: $destination。请换一个新的 Label。"
14        return
15    }
16
17    New-Item -ItemType Directory -Force -Path $destination | Out-Null
18    Get-ChildItem -Path $project -Force | Copy-Item -Destination $destination -
19    Recurse -Force
20    Write-Host "快照完成: $destination"
21 }
```

第 3 步: 创建第一个快照

在同一个终端输入:

代码块New-TrainingSnapshot "step-00-empty-project"

你应该看到什么

终端会显示类似：

代码块

```
1 快照完成：C:\Users\你的用户名\Documents\CodexTraining\snapshots\step-00-empty-project
```

第 4 步：确认快照真的存在

运行：

代码块

```
1  Get-ChildItem "$HOME\Documents\CodexTraining\snapshots\step-00-empty-project"
```

应该能看到 `notes.md`。

重要： 这个函数仅在当前终端窗口有效。关闭终端后要重新粘贴一次；也可以把它保存为自己的 PowerShell profile，但本教程先不增加这个复杂度。

4.4 本教程的快照规则

后续每次看到“创建快照”时，在终端运行：

代码块

```
1  New-TrainingSnapshot "这里替换成步骤名称"
```

建议使用下面的命名：

修改前的节点	快照名称
项目刚生成	step-02-baseline
加关键词搜索前	step-04-before-search
加截止日期前	step-06-before-due-date
重构前	step-07-before-refactor

5. 实验 1：只读分析与精确提供上下文

目标： 练习“先问清楚，再动手”，以及给 Codex 精准上下文。

目前项目里只有 `notes.md`。这正适合练只读分析：如果 Codex 把不存在的 Java 架构讲得很复杂，说明它在猜。

5.1 只读分析空项目

第 1 步：打开 Codex 面板

点击 VS Code 侧边栏的 Codex 图标。

第 2 步：输入下面提示词

代码块

```
1  请只读检查当前打开的项目文件夹，不要创建、修改或删除任何文件，不要运行命令。
2
3  请用中文回答：
4  1. 当前有哪些文件；
5  2. 这个项目是否已经是 Java/Maven 项目；
6  3. 接下来要创建一个可运行 Task Tracker，最少需要哪些文件；
7  4. 哪些信息目前还无法从现有文件确定。
```

第 3 步：检查回答是否可靠

好的回答应包括：

- 当前只有 `notes.md`。
- 当前还不是 Java/Maven 项目。
- 需要 `pom.xml`、`src/main/java`、`src/test/java` 等结构。
- 不能假装已经存在 `TaskService` 或测试。

不好的回答

- “当前 `TaskService` 已使用某种模式……” 但项目里根本没有这个文件。
- 没有被要求就创建一堆文件。
- 没有解释自己的不确定性。

5.2 用 @文件 提供上下文

第 1 步：在 Codex 输入框输入

代码块

1 请解释 @notes.md 的内容，并指出其中哪两条规则会影响后面的 Codex 工作方式。不要修改文件。

第 2 步：从自动补全中选择 notes.md

不要只手打文件名；要从出现的文件列表里点击正确的 notes.md。

通过标准

Codex 能指出：

- 每次大改前创建本地快照。
- 修改后运行 `mvn test`。

5.3 用“选中代码”提供上下文

1. 在 Explorer 中打开 notes.md。
2. 选中其中的三条列表。
3. 右键或使用编辑器里出现的 Codex 操作入口。
4. 提问：

代码块

1 请把选中的规则改写成适合写给编程助手的三条明确约束。先只给文字建议，不要直接编辑文件。

你在练什么

- 不要让 Codex 扫描所有文件再猜。
- 一段明确的选中内容，通常比“帮我看一下项目”更高效。
- 先让它解释和建议，再选择是否应用改动。

6. 实验 2：从需求到可运行 Java 项目

目标：练习完整流程：需求 → 计划 → 审阅计划 → 实施 → 测试 → 本地快照。

6.1 先创建修改前快照

确认前面定义快照函数的终端还开着，然后运行：

代码块

```
1 New-TrainingSnapshot "step-02-baseline"
```

若提示函数不存在，重新粘贴 4.3 的函数定义，再运行上面的命令。

6.2 先让 Codex 出计划，不让它改文件

在 Codex 面板中，若界面有 **Plan** 或 **Plan mode**，先切换到它。若没有看到，直接在提示词里明确要求“先计划”。

发送：

代码块

```
1 先只输出实施计划，不要创建、修改或删除文件，不要运行命令。
2
3 目标：在当前空项目中创建一个可运行的 Java 17 + Maven 命令行 Task Tracker。
4
5 固定要求：
6 - Maven 坐标：com.janel.codexlab:task-tracker;
7 - package: com.janel.codexlab;
8 - 只使用 Java 标准库与 JUnit 5;
9 - Task 至少有 id、title、completed 三个字段;
10 - TaskService 至少支持 addTask、listTasks、completeTask;
11 - App 演示创建两条任务、完成其中一条、打印结果;
12 - 至少有 3 个 JUnit 测试;
13 - 完成后必须运行 mvn test;
14 - 不使用 Git 或 GitHub;
15 - 不要加入数据库、网页界面、Spring、Lombok 或其他无关依赖。
16
17 请按“文件清单 → 每个文件职责 → 实施顺序 → 测试点 → 风险”输出计划。
```

6.3 人工审阅计划

确认计划至少包含：

- pom.xml
- src/main/java/com/janel/codexlab/App.java
- Task.java

- `TaskService.java`
- 至少一个 `src/test/java/...` 测试文件
- `mvn test`

计划过大时，发送这条

代码块

- 1 计划范围太大。请删掉任何数据库、Web、持久化、用户系统、第三方框架和非必要抽象，只保留内存中的命令行待办程序与 JUnit 测试。

6.4 允许实施

计划合格后，发送：

代码块

- 1 计划可以。请按计划实施。
- 2
- 3 约束：
 - 4 - 只修改当前 `task-tracker` 文件夹；
 - 5 - 只使用 Java 17、Maven、JUnit 5；
 - 6 - 不安装无关依赖；
 - 7 - 每完成一个关键文件，用一句话说明它的职责；
 - 8 - 最后运行 `mvn test`；
 - 9 - 最后列出：创建或修改了哪些文件、测试结果、我应该手动检查什么；
- 10 - 不使用 Git 或 GitHub。

6.5 查看 Codex 的文件改动请求

Codex 可能逐个显示修改、执行命令或请求批准。

你要做的：

1. 看它要写入的路径是否都在 `task-tracker` 下。
2. 看 `pom.xml` 是否只有合理依赖。
3. 看到 `mvn test`，可以批准。
4. 看到 `Remove-Item`、联网下载不明脚本、修改用户目录，不要批准。

6.6 亲自运行构建和测试

无论 Codex 是否说“已通过”，都在 VS Code 终端手动运行：

```
1 mvn test
2 mvn package
3 java -cp target/classes com.janel.codexlab.App
```

预期结果

- `mvn test` 最终显示 `BUILD SUCCESS`。
- `mvn package` 最终显示 `BUILD SUCCESS`。
- 最后一条命令打印至少两条任务，其中一条状态已完成。

如果 `java -cp` 报找不到主类

先看实际文件路径：

代码块

```
1 Get-ChildItem -Recurse src\main\java
```

然后问 Codex：

代码块

```
1 我运行 `java -cp target/classes com.janel.codexlab.App` 失败，下面是完整终端输出：
2
3 <把完整报错粘贴这里>
4
5 请只分析失败原因、实际主类名和应使用的运行命令。不要修改文件。
```

6.7 创建可运行基线快照

测试通过后运行：

代码块

```
1 New-TrainingSnapshot "step-02-running-baseline"
```

实验验收

- ☐ 项目中有 pom.xml。
- ☐ 有 App、Task、TaskService。
- ☐ 有 JUnit 测试。

- ☐ mvn test 通过。
- ☐ 已创建 step-02-running-baseline 快照。

7. 实验 3：读懂 Codex 生成的项目

目标：不只是“让 Codex 写完”，而是你能解释项目为什么这样组织。

7.1 让 Codex 画项目地图

在 Codex 面板发送：

代码块

```
1  请只读分析当前 Task Tracker 项目，不要修改文件，不要运行命令。
2
3  请用中文输出：
4  1. 文件树（只列出关键文件）；
5  2. 程序从 App 启动后的调用顺序；
6  3. Task、TaskService、测试类分别负责什么；
7  4. 一条任务从“创建”到“完成”时数据如何变化；
8  5. 当前设计的两个局限。
9
10 要求：每个结论都要标明对应的文件名和方法名。
```

7.2 打开核心文件逐个核对

在 Explorer 依次打开：

1. App.java
2. Task.java
3. TaskService.java
4. 测试文件

每看一个文件，问自己：

- 它的输入是什么？
- 它的输出或状态变化是什么？
- 业务逻辑有没有放错地方？
- 测试是否覆盖了正常情况和异常情况？

7.3 用一个小问题检验理解

发送：

代码块

- 1 请只读回答：如果我执行 `completeTask(999)`，而 `999` 不存在，当前项目会发生什么？
- 2
- 3 请先定位相关方法，再说明实际行为、对应测试是否覆盖、以及你建议的行为。不要修改文件。

你应该学到

- Codex 的回答必须基于现有代码，不是基于“通常应该怎样”。
- “实际行为”和“建议行为”是两件不同的事。
- 当需要改需求时，先写测试或先制定明确验收标准。

8. 实验 4：新增小功能，练习测试优先

目标：为 Task Tracker 增加“按关键词搜索任务”，并练习先定义行为、再让 Codex 写代码。

8.1 创建快照

代码块

- 1 `New-TrainingSnapshot "step-04-before-search"`

8.2 先定义验收标准

在 `notes.md` 末尾手动增加：

代码块

- 1 `## 功能：关键词搜索`
- 2
- 3 `- 输入关键词后，返回 title 中包含该关键词的任务。`
- 4 `- 搜索忽略大小写。`
- 5 `- 返回结果保留原来的任务顺序。`
- 6 `- 空关键词返回所有任务。`
- 7 `- 关键词没有匹配时返回空列表，不抛异常。`

保存文件。

8.3 让 Codex 先写测试设计

发送：

代码块

```
1  请只读阅读 @notes.md、Task.java、TaskService.java 和现有测试文件。
2
3  现在要实现“关键词搜索”功能。先不要修改代码。
4
5  请给出：
6  1. 建议新增的方法签名；
7  2. 至少 4 个 JUnit 测试案例；
8  3. 每个测试验证什么；
9  4. 哪些现有文件需要改动；
10 5. 是否需要修改 App，为什么。
```

8.4 让 Codex 先写失败测试

审阅测试设计后发送：

代码块

```
1  请先只新增关键词搜索的 JUnit 测试，不修改生产代码。
2
3  要求覆盖：
4  - 大小写不敏感；
5  - 子串匹配；
6  - 空关键词；
7  - 无匹配结果；
8  - 原始任务顺序。
9
10 完成后运行 mvn test，并报告哪些测试因为功能尚未实现而失败。不要修改其他无关文件。
```

预期结果

`mvn test` 应该失败，失败原因通常是搜索方法不存在或尚未实现。这是正常的。

8.5 再实现最小功能

发送：

代码块

```
1  请根据刚才新增的失败测试，实现最小的关键词搜索功能。
2
3  约束：
4  - 不新增第三方依赖；
```

- 5 - 不改变已有 add、list、complete 的行为；
- 6 - 不改无关文件；
- 7 - 最后运行 mvn test；
- 8 - 在回答中说明搜索大小写不敏感的实现方式。

8.6 手动验证

在终端运行：

代码块

```
1 mvn test
```

测试通过后，问 Codex：

代码块

- 1 请只读检查关键词搜索实现。请说明：
- 2 1. 搜索方法在哪个类；
- 3 2. 空关键词为什么会返回全部；
- 4 3. 为什么结果顺序不会改变；
- 5 4. 哪个测试覆盖了大小写不敏感。
- 6 不要修改文件。

8.7 创建新快照

代码块

```
1 New-TrainingSnapshot "step-04-search-complete"
```

9. 实验 5：故意制造 bug，再让 Codex 用证据修复

目标：学会提供完整报错、先定位根因、要求最小修改，而不是看到失败就让 Codex “随便修”。

9.1 创建快照

代码块

```
1 New-TrainingSnapshot "step-05-before-debugging"
```

9.2 让 Codex 故意引入可控 bug

发送：

代码块

- 1 为了练习调试，请在当前关键词搜索实现中故意引入一个简单、可复现的 bug：把“子串匹配”错误改成“完全相等匹配”。
- 2
- 3 约束：
- 4 - 只修改生产代码中的最少位置；
- 5 - 不修改任何测试；
- 6 - 不修复它；
- 7 - 修改后只说明改了哪个文件和哪个方法。

9.3 亲自运行测试并复制完整输出

终端运行：

代码块

```
1 mvn test
```

失败后，不要只复制最后一句。尽量复制包含下列内容的部分：

- 哪个测试失败；
- expected 和 actual；
- 类名、方法名、行号；
- Maven 最后的失败摘要。

9.4 让 Codex 先诊断，不要急着修

发送：

代码块

- 1 下面是 mvn test 的完整失败输出：
- 2
- 3 <把终端输出完整粘贴在这里>
- 4
- 5 请先不要修改文件。请按以下格式回答：
- 6 1. 失败的是哪个测试；
- 7 2. 测试原本期待的行为；
- 8 3. 生产代码的实际行为；
- 9 4. 最可能的根因，精确到文件和方法；

- 10 5. 最小修复方案；
- 11 6. 修复后应该再运行哪些验证。

9.5 允许最小修复

如果诊断准确，发送：

代码块

- 1 请实施刚才的最小修复。
- 2
- 3 限制：
- 4 - 只修改根因所在的代码；
- 5 - 不删除或弱化失败测试；
- 6 - 不重构无关代码；
- 7 - 运行 `mvn test`；
- 8 - 最后说明修改了什么，以及为什么不会影响空关键词和大小写不敏感。

9.6 最终验证与快照

代码块

- 1 `mvn test`
- 2 `New-TrainingSnapshot "step-05-debugging-complete"`

你在练什么

正确的调试顺序是：

代码块

- 1 复现失败 → 保存完整证据 → 定位根因 → 最小修复 → 运行测试 → 验证边界行为

不是：

代码块

- 1 看到报错 → 让 AI 大改一堆代码 → 测试偶然通过 → 不知道改了什么

10. 实验 6：多文件功能与 Plan mode

目标： 为任务增加可选截止日期，并练习在复杂任务前先让 Codex 写计划。

10.1 创建快照

代码块

```
1 New-TrainingSnapshot "step-06-before-due-date"
```

10.2 明确本次需求

在 `notes.md` 增加：

代码块

```
1 ## 功能：截止日期
2
3 - 每个任务可以没有截止日期，也可以有一个 LocalDate 类型的截止日期。
4 - 任务创建时可以传入截止日期。
5 - 已完成任务不属于逾期。
6 - 未完成且 dueDate 早于今天的任务属于逾期。
7 - dueDate 等于今天不算逾期。
8 - 提供 findOverdueTasks(LocalDate today) 方法。
9 - 必须给出明确的 JUnit 边界测试。
```

保存。

10.3 请求计划

发送：

代码块

```
1 请阅读 @notes.md、当前 Task、TaskService、App 和测试文件。
2
3 先只输出实施计划，不要修改文件、不要运行命令。
4
5 目标：为任务增加可选截止日期，并实现 findOverdueTasks(LocalDate today)。
6
7 请务必包含：
8 1. 需要改哪些文件；
9 2. Task 的字段或构造方式如何变化；
10 3. 如何避免破坏旧调用；
11 4. 逾期判断的精确条件；
```

- 12 5. 要新增哪些测试，尤其包括“昨天、今天、明天、已完成、无截止日期”；
- 13 6. App 是否需要演示该功能；
- 14 7. 实施后的验证命令。

10.4 审查计划时重点看什么

确认它没有：

- 把 `LocalDate` 存成字符串；
- 用系统当前日期硬编码在业务方法里；
- 把“今天”算成逾期；
- 把已完成任务算成逾期；
- 无理由修改所有旧接口。

若出现问题，发送：

代码块

- 1 请修正计划：截止日期必须在 Java 中使用 `LocalDate`；逾期判断必须通过方法参数 `today` 注入；等于 `today` 不算逾期；不要加入数据库或外部库。

10.5 先写测试，再实现

发送：

代码块

- 1 请按已确认的计划实施，但遵守以下顺序：
- 2
- 3 1. 先新增或更新 JUnit 测试；
- 4 2. 运行 `mvn test`，报告预期失败；
- 5 3. 再实现最小生产代码；
- 6 4. 重新运行 `mvn test`；
- 7 5. 只修改实现此功能所需的文件；
- 8 6. 最后逐项对照需求说明是否满足。

10.6 手动运行和查看输出

代码块

- 1 `mvn test`
- 2 `mvn package`

```
3 java -cp target/classes com.janel.codexlab.App
```

如果 App 没有展示截止日期功能，不是错误；它是否展示应按你在计划阶段做的决定。关键是测试与业务规则正确。

10.7 建立新快照

代码块

```
1 New-TrainingSnapshot "step-06-due-date-complete"
```

11. 实验 7：重构而不改变行为

目标： 练习“先锁住行为，再改善代码结构”。

11.1 创建快照

代码块

```
1 New-TrainingSnapshot "step-07-before-refactor"
```

11.2 先让 Codex 找重复逻辑

发送：

代码块

```
1 请只读分析当前项目，不要修改文件，不要运行命令。
2
3 请找出与“任务标题合法性”或“输入校验”有关的重复逻辑。
4
5 请输出：
6 1. 重复逻辑分别出现在哪些文件和方法；
7 2. 这些重复是否已经造成 bug；
8 3. 一个最小的重构方案；
9 4. 哪些现有测试能够证明重构前后行为相同；
10 5. 这次不应该碰哪些文件。
```

11.3 先写重构计划

发送：

代码块

- ```
1 根据刚才的分析，请只输出一个最小重构计划。不要修改文件。
2
3 限制：
4 - 不新增依赖；
5 - 不改公开方法的语义；
6 - 不调整 package；
7 - 尽量只新增一个私有辅助方法或一个很小的校验类；
8 - 不更改 App 输出格式，除非测试或需求要求；
9 - 必须保留并运行现有测试。
```

## 11.4 实施重构

发送：

代码块

- ```
1  请实施刚才确认的最小重构。
2
3  要求：
4  - 先说明你会改哪几个文件；
5  - 完成后运行 mvn test；
6  - 列出修改的每个方法；
7  - 说明为什么这是重构而不是新增行为；
8  - 不格式化无关文件，不做额外清理。
```

11.5 用测试确认行为没有变化

在终端运行：

代码块

- ```
1 mvn test
```

若通过，再问：

代码块

- ```
1  请只读回答：这次重构前后，哪些输入的行为应该完全相同？请举 3 个来自现有测试的例子，并说明
    对应测试方法名。不要修改文件。
```

11.6 创建快照

代码块

```
1 New-TrainingSnapshot "step-07-refactor-complete"
```

12. 实验 8：本地代码审查与文件比较

目标： 没有 Git/GitHub 时，仍然可以系统地检查 Codex 改动。

12.1 让 Codex 只读审查

发送：

代码块

```
1 请只读审查当前 Task Tracker 项目，不要修改文件，不要运行命令。
2
3 优先检查：
4 1. 空标题、空白标题、null 标题是否有明确行为；
5 2. 不存在的任务 id 会怎样；
6 3. 搜索的空关键词和大小写规则是否一致；
7 4. 逾期逻辑在昨天、今天、明天、已完成、无截止日期时是否正确；
8 5. 测试是否缺少重要边界。
9
10 只报告高置信度问题。每个问题必须包含：文件名、方法名、触发条件、影响、建议的最小修复。若没
    有高置信度问题，请明确说“未发现高置信度问题”。
```

12.2 不要让 Codex 一键修所有建议

对每一条发现问自己：

- 它真的和当前需求有关吗？
- 它是否有测试或代码证据？
- 修复会不会扩大范围？
- 我是否理解改动？

只选择你认同的一条，例如：

代码块

```
1 我只接受第 2 条：<复制那条问题的简短描述>。
```

2

- 3 请只修复这个问题，并新增一个能失败再通过的测试。不要处理其他建议，不要重构无关代码。完成后运行 `mvn test`。

12.3 使用 VS Code 比较“快照文件”和“当前文件”

假设你要对比重构前后的 `TaskService.java`。

方法 A：使用命令打开差异对比

在 PowerShell 中运行。路径可按实际项目结构调整：

代码块

```
1 code --diff `
2     "$HOME\Documents\CodexTraining\snapshots\step-07-before-
   refactor\src\main\java\com\janel\codexlab\TaskService.java" `
3     "$HOME\Documents\CodexTraining\task-
   tracker\src\main\java\com\janel\codexlab\TaskService.java"
```

VS Code 会打开左右对比窗口：

- 左侧：修改前快照。
- 右侧：当前版本。
- 高亮处：不同内容。

方法 B：从 Explorer 手动比较

1. 用资源管理器打开 `Documents\CodexTraining\snapshots`。
2. 找到需要的快照。
3. 在 VS Code 中打开快照里的文件和当前项目中的同名文件。
4. 在第一个文件的标签页上右键，选择 **Select for Compare**。
5. 在第二个文件的标签页上右键，选择 **Compare with Selected**。

12.4 你需要检查的“坏改动”信号

- 一个很小功能却修改了许多无关文件。
- Codex 改掉或删除了原有测试，只为了让测试通过。
- 新增了不必要依赖。
- 变量名、格式、注释大范围变化但没有业务收益。
- 出现你没有请求的联网、文件删除或配置变更。

12.5 创建审查后快照

代码块

```
1 New-TrainingSnapshot "step-08-reviewed"
```

13. 实验 9：补 README 与交付说明

目标： 让 Codex 帮你写文档，但每个命令都必须真实可执行。

13.1 创建快照

代码块

```
1 New-TrainingSnapshot "step-09-before-readme"
```

13.2 先让 Codex 收集事实

发送：

代码块

```
1 请只读分析当前项目，不要修改文件，不要运行命令。
2
3 请收集写 README 所需的事实：
4 1. 项目用途；
5 2. Java 和 Maven 要求；
6 3. 实际 package 与主类；
7 4. 实际可运行的测试、打包、运行命令；
8 5. 当前支持的功能；
9 6. 当前明确不支持的功能；
10 7. 现有测试覆盖的主要场景。
11
12 不要猜测没有证据的功能。
```

13.3 让 Codex 起草 README

发送：

代码块

```
1  请基于刚才确认的事实，新建或更新 README.md。
2
3  README 必须包含：
4  - 项目简介；
5  - 功能列表；
6  - 环境要求；
7  - 从项目根目录运行测试、打包、运行程序的命令；
8  - 一个简短示例输出；
9  - 已知限制；
10 - 不要写任何 Git/GitHub、远程仓库、部署或数据库内容；
11 - 不要声称未实现的功能。
12
13 完成后只告诉我 README 写入了什么；不要修改 Java 代码。
```

13.4 验证 README 中每条命令

打开 `README.md`，把出现的命令逐条在终端运行。

至少确认：

代码块

```
1  mvn test
2  mvn package
3  java -cp target/classes com.janel.codexlab.App
```

README 命令与实际不同时

不要手动猜。发送：

代码块

```
1  README 中的这条命令无法运行：
2
3  <粘贴命令和完整报错>
4
5  请只修正 README 中这条命令，保持其他内容不变。不要修改 Java 或 Maven 文件。
```

13.5 创建快照

代码块

```
1  New-TrainingSnapshot "step-09-readme-complete"
```

14. 实验 10：用 AGENTS.md 固化项目规则

目标： 把你每次都会重复说的规则放到 `AGENTS.md`，让 Codex 在当前项目中有稳定的工作约束。

`AGENTS.md` 是给 Codex 看的项目指引文件。它应简短、准确、可执行；不要写成长篇“开发哲学”。

14.1 创建 `AGENTS.md`

在项目根目录新建：

代码块

```
1 AGENTS.md
```

粘贴以下内容：

代码块

```
1 # Task Tracker 工作规则
2
3 ## 项目目的
4 这是一个用于学习 Codex 工作流的 Java 17 + Maven 命令行 Task Tracker。
5
6 ## 基本命令
7 - 测试：`mvn test`
8 - 打包：`mvn package`
9 - 运行：`java -cp target/classes com.janel.codexlab.App`
10
11 ## 编码规则
12 - 使用 Java 17。
13 - 生产代码位于 `src/main/java/com/janel/codexlab/`。
14 - 测试位于 `src/test/java/com/janel/codexlab/`。
15 - 不加入数据库、Web 框架、Lombok 或无关第三方依赖。
16 - 尽量做小而聚焦的修改。
17 - 不改无关文件，不做全项目格式化。
18
19 ## 验证规则
20 - 修改 Java 代码后，运行 `mvn test`。
21 - 新增行为时，优先新增或更新 JUnit 测试。
22 - 遇到需求不明确时，先提问或先给出计划，不要自行扩展范围。
23
24 ## 文件安全规则
25 - 只操作当前 task-tracker 项目文件夹。
```

- 26 - 不删除文件，除非用户明确同意。
- 27 - 不使用 Git 或 GitHub，也不创建远程仓库。

保存。

14.2 测试 Codex 是否读取规则

在 Codex 面板新开一个对话，发送：

代码块

- 1 请先读取项目根目录的 AGENTS.md，然后只用 6 条要点总结你在这个项目里必须遵守的规则。不要修改文件，不要运行命令。

通过标准

回答中应出现：

- Java 17。
- `mvn test`。
- 小而聚焦的修改。
- 不加无关依赖。
- 不删除文件。
- 不使用 Git 或 GitHub。

14.3 检查规则是否真的影响行为

再发送：

代码块

- 1 请为 Task Tracker 增加一个“按 id 删除任务”的功能。
- 2
- 3 先只输出计划。请明确说明你会如何遵守 AGENTS.md 中的测试、依赖、文件安全和范围控制规则。不要修改文件。

如果它建议删掉 `TaskService.java` 重写、添加数据库或大型依赖，说明计划不合格；要求它缩小范围。

15. 实验 11：Codex CLI：交互、一次性提问、自动化输出

目标： 在终端中使用 Codex。CLI 适合：你已经知道要问什么、需要把结果保存成文件，或想把同类检查重复执行。

本实验可选。前面的 VS Code 流程已经足够完成日常学习。

15.1 安装并检查 CLI

第 1 步：打开 PowerShell

建议使用 Windows Terminal 或 VS Code 终端。

第 2 步：安装 Codex CLI

官方 Windows 安装方式会下载并执行官方安装器。确认你使用的是官方命令后再运行：

代码块

```
1 powershell -ExecutionPolicy ByPass -c "irm  
https://chatgpt.com/codex/install.ps1 | iex"
```

也可以在你已经安装 Node.js 的情况下使用：

代码块

```
1 npm install -g @openai/codex
```

第 3 步：确认命令可用

代码块

```
1 codex --version
```

第 4 步：登录

代码块

```
1 codex login
```

选择 ChatGPT 登录，并在浏览器完成授权。

15.2 CLI 中的只读项目分析

切换到项目目录：

代码块

```
1 Set-Location "$HOME\Documents\CodexTraining\task-tracker"
```

运行一次性只读任务：

代码块

```
1 codex exec --skip-git-repo-check --sandbox read-only "只读分析当前 Java 项目。用不  
超过 10 行中文说明：项目入口、核心类、测试命令、当前支持的功能和两个设计限制。不要修改文  
件，不要运行会写入文件的命令。"
```

你应该看到什么

- CLI 在当前项目目录读取文件。
- 它输出项目总结。
- 不生成或修改项目文件。

`--skip-git-repo-check` 让 `codex exec` 可以在非版本控制目录中运行；`--sandbox read-only` 限制这次任务只读。

15.3 启动交互式 CLI 会话

先运行：

代码块

```
1 codex
```

若它提示当前文件夹不是版本控制项目，不要为了消除这个提示去安装或配置 Git；继续使用上一节的 `codex exec --skip-git-repo-check` 完成这份教程即可。

若交互式界面正常打开，输入：

代码块

```
1 只读解释当前项目中 TaskService 的职责、公开方法、状态如何保存、测试如何覆盖它。不要修改文  
件。
```

结束会话通常可输入 `/exit`，或者按 `Ctrl + C`。

15.4 将一次性分析保存为报告文件

在项目根目录运行：

代码块

```
1 codex exec --skip-git-repo-check --sandbox read-only --output-last-message local-architecture-report.md "只读分析当前项目。生成一份中文架构说明，包含文件职责、调用关系、业务规则、测试覆盖、风险点。不要修改现有文件。"
```

然后查看文件：

代码块

```
1 Get-Content .\local-architecture-report.md
```

注意

这条命令会新建 `local-architecture-report.md`。它没有修改 Java 代码，但仍然是写文件动作。先确认你接受新报告文件再运行。

15.5 使用 `codex exec` 自动做一次只读质量检查

在项目根目录运行：

代码块

```
1 codex exec --skip-git-repo-check --sandbox read-only --output-last-message local-quality-report.md "只读审查当前 Java Maven 项目。重点检查：输入校验、空值、id 不存在、关键词搜索、逾期边界、测试缺口。仅报告高置信度问题；每条都要有文件名、方法名、影响和最小修复建议。不要修改现有文件。"
```

打开 `local-quality-report.md`，选择一条你认同的问题，再回到 VS Code Codex 面板完成“最小修复 + 测试”。

15.6 CLI 出问题时

问题	先做什么
<code>codex is not recognized</code>	关闭并重新打开 PowerShell；再执行 <code>codex --version</code>
登录失败	运行 <code>codex logout</code> 后再 <code>codex login</code>
提示目录不是版本控制项目	对 <code>codex exec</code> 加 <code>--skip-git-repo-check</code>

不小心选择了可写权限	退出会话，重新用 <code>--sandbox read-only</code> 执行
CLI 无法读取项目目录	确认当前目录是项目根目录；必要时在 Codex 中使用对应的读目录授权命令

16. 实验 12：Codex App：并行线程与只读自动化

目标： 学会在 Windows Codex App 中查看多个任务、审查结果，以及用低风险方式安排重复检查。

这里不使用工作树、分支或远程仓库。非版本控制项目中的自动化会直接在项目目录运行，因此本教程只建议创建只读自动化。

16.1 安装 Codex App（可选）

在 Windows 上可以从 Microsoft Store 安装 Codex App，也可以运行：

代码块

```
1 winget install Codex -s msstore
```

打开后，使用 ChatGPT 账号登录。

16.2 添加本地项目

1. 打开 Codex App。
2. 选择 **Add new project** 或类似入口。
3. 选择你的项目目录：

代码块

```
1 Documents\CodexTraining\task-tracker
```

4. 选择 **Local**，表示让 Codex 在你的电脑本地项目中工作。
5. 确认权限保持默认安全设置，不选择 Full Access。

16.3 建立两个“只读并行线程”

在同一项目中分别开两个线程：

线程 A：架构说明

代码块
只读分析当前项目。输出一张表：文件、职责、关键方法、被谁调用。不要修改文件，不要运行写操作。

线程 B：测试审查

代码块

- 1 只读分析当前测试。列出每个测试覆盖的规则、缺少的边界、以及最值得增加的一条测试。不要修改文件，不要运行写操作。

关键规则：同一时间只允许一个“写入线程”

如果你之后让其中一个线程修改代码：

1. 先确保另一个线程只读，不要同时写同一目录。
2. 修改前手动创建快照。
3. 修改后运行 `mvn test`。
4. 在 App 的 review/变更区查看它改了些什么。

没有工作树隔离时，两个写入线程同时操作同一项目容易互相覆盖，因此不要这样做。

16.4 创建只读自动化：每日测试检查（可选）

先在普通线程中测试这条提示词：

代码块

- 1 只运行 `mvn test`，并报告：测试是否通过、失败测试的名称、以及最有用的下一步。不要修改任何文件，不要访问网络。

手动测试正确后，再创建 Automation：

1. 打开 App 的 **Automations** 区域。
2. 新建一个项目自动化。
3. 名称可写： `Task Tracker Daily Test`。
4. 选择每天一次或你想要的频率。
5. 粘贴上面的只读提示词。
6. 确认它使用默认安全权限或只读权限。
7. 不允许它修改文件。

自动化验收

- 它能运行测试并把结果汇总到 App 的结果/收件箱区域。
- 失败时，它报告失败，不自行改代码。
- 通过时，它只报告结果。

17. 实验 13：创建个人 Skill

目标： 把“检查 Java 质量”的固定步骤打包为你个人可复用的 Skill，而不是放到项目目录或任何在线平台。

Skills 可在 CLI、IDE 扩展和 Codex App 中使用。个人 Skills 可以保存在 Windows 用户目录下的 `.agents\skills`，适用于你自己的本机环境。

17.1 创建 Skill 文件夹

PowerShell 中运行：

代码块

```
1 $skillDir = Join-Path $HOME ".agents\skills\java-quality-check"
2 New-Item -ItemType Directory -Force -Path $skillDir | Out-Null
3 code (Join-Path $skillDir "SKILL.md")
```

17.2 写入 `SKILL.md`

在打开的文件中粘贴：

代码块

```
1 ---
2 name: java-quality-check
3 description: Review a small Java Maven project for correctness, tests, input
  validation, and edge cases. Use when the user asks to inspect Java code
  quality, verify tests, or review a small change. Do not use for large feature
  design or unrelated writing tasks.
4 ---
5
6 # Java Quality Check
7
8 ## 目标
9 对当前 Java Maven 项目做只读质量检查，并给出最小、可验证的建议。
10
11 ## 步骤
12 1. 先阅读 `AGENTS.md`（如果存在）。
13 2. 找到 `pom.xml`、主代码目录和测试目录。
```

```
14 3. 说明实际测试命令；只有在用户允许时才运行测试。
15 4. 检查输入校验、null、空字符串、边界日期、集合顺序、id 不存在等情况。
16 5. 只报告高置信度问题。
17 6. 每条问题必须给出：文件、方法、触发条件、影响、最小修复、建议测试。
18 7. 默认不修改文件；只有用户明确要求后才实施修复。
19
20 ## 不要做
21 - 不新增依赖。
22 - 不改无关文件。
23 - 不做大范围格式化。
24 - 不删除测试。
25 - 不使用 Git 或 GitHub。
```

保存文件。

17.3 重启或重新打开 Codex

Codex 通常会自动发现 Skill 变化。若没有出现：

1. 关闭并重新打开 VS Code 的 Codex 面板，或重启 Codex App/CLI。
2. 在 CLI 或 IDE 中输入 `/skills`，查看是否能看到 `java-quality-check`。

17.4 调用 Skill

在 Codex 面板尝试：

代码块

```
1 请使用 $java-quality-check 只读审查当前 Task Tracker 项目。不要修改文件。
```

通过标准

- 它按 Skill 的步骤检查项目。
- 它只报告高置信度问题。
- 它不会自行开始修改。

17.5 为什么个人 Skill 比项目 Skill 更适合这份教程

你现在不使用版本控制，所以将 Skill 放在 `$HOME\.agents\skills`：

- 不需要任何仓库结构。
- 所有本地 Java 练习项目都可复用。
- 不会把教程复杂度扩展到团队协作、共享分发或远程管理。

18. 实验 14：MCP 的安全入门

目标： 理解 MCP 是什么、什么时候该用、什么时候不要用。此实验默认不连接任何外部服务。

18.1 MCP 是什么

MCP (Model Context Protocol) 可以让 Codex 使用外部工具或数据源，例如文档系统、任务系统、浏览器、数据库或内部服务。

对初学者来说，它的价值不是“能连得越多越好”，而是：当你有一个**重复、明确、可信**的外部信息来源时，Codex 才能通过 MCP 更可靠地使用它。

18.2 什么时候不需要 MCP

当前的 Task Tracker 项目不需要 MCP，因为：

- 所有代码都在本地文件夹。
- 所有验证都能用 `mvn test` 完成。
- 没有外部文档、数据库或工单系统需要读取。
- 接入外部工具会增加权限与隐私风险。

18.3 在连接 MCP 前必须回答的五个问题

1. 这个 MCP 连接的是什么服务？
2. 它能读什么数据？
3. 它能写什么数据？
4. 它是否需要 token、登录或浏览器授权？
5. 不接它时，Codex 是否仍能完成任务？

第 5 题答案是“能”时，先不要接。

18.4 安全练习：让 Codex 设计而不配置

在 Codex 中发送：

代码块

- 1 假设未来这个项目需要读取一个只读的内部 API 文档服务。请不要配置任何 MCP，也不要生成 token。
- 2
- 3 请只输出：
- 4 1. 是否真的需要 MCP；

- 5 2. 最小权限应该是什么；
- 6 3. 哪些数据绝不能授权；
- 7 4. 如何先在隔离测试项目验证；
- 8 5. 什么情况下应该停止接入。

通过标准

好的回答会强调最小权限、只读、先小范围试用、不要把密钥写进代码或项目文件。

19. 迁移到你的真实 Java 项目

当你要把这套流程用到 CSE 作业、个人项目或新的 Java 程序时，按照这个顺序做。

19.1 第一次接触陌生项目

1. 先复制项目文件夹，保留一份原始副本。
2. 用 VS Code 打开复制后的项目。
3. 让 Codex 只读输出：文件树、构建命令、测试命令、入口、风险。
4. 亲自运行测试，确认基线状态。
5. 写或更新 `AGENTS.md`，告诉 Codex 不应该触碰什么。

19.2 要加一个功能时

1. 在项目外侧创建本地快照。
2. 写清楚验收标准。
3. 要求 Codex 先计划、不改文件。
4. 审查计划。
5. 要求先写或更新测试。
6. 再写最小实现。
7. 运行测试。
8. 用 VS Code 比较快照和当前关键文件。
9. 创建新的“功能完成”快照。

19.3 遇到 bug 时

1. 先复现。
2. 复制完整错误信息。

3. 让 Codex 先诊断，不要立刻修改。
4. 让它提出最小修复。
5. 修复后运行原始失败测试和全部测试。
6. 保留修复前快照，方便你复盘。

19.4 不应交给 Codex 自行决定的事

- 课程作业是否符合学术诚信要求。
 - 删除大量文件、替换整个项目。
 - 添加不理解的依赖。
 - 修改密码、密钥、个人信息或登录配置。
 - 把本地代码上传到任何外部服务。
 - 用“测试通过”代替你对设计的理解。
-

20. 常用提示词模板

下面模板直接复制后替换尖括号内容即可。

20.1 只读理解项目

代码块

```
1 只读分析当前项目，不要修改文件，不要运行命令。
2
3 目标：<我想理解的问题>
4 请输出：
5 1. 相关文件和方法；
6 2. 当前实际行为；
7 3. 证据（文件名、方法名或测试名）；
8 4. 不确定的地方；
9 5. 下一步最小建议。
```

20.2 小功能：先计划

代码块

```
1 先只输出实施计划，不要修改文件、不要运行命令。
2
3 需求：<功能描述>
4 验收标准：
```

```
5 - <标准 1>
6 - <标准 2>
7 - <边界情况>
8
9 约束：Java 17；Maven；不加依赖；不改无关文件；不使用 Git 或 GitHub。
10
11 请给出：文件清单、方法变化、测试方案、实施顺序、风险与验证命令。
```

20.3 小功能：测试优先实施

代码块

```
1 请按确认的计划实施。
2
3 顺序必须是：
4 1. 先新增或更新测试；
5 2. 运行 mvn test 并说明失败原因；
6 3. 实现最小生产代码；
7 4. 再次运行 mvn test；
8 5. 列出修改文件、测试结果、剩余风险。
9
10 不要加依赖，不要改无关文件，不要删除测试。
```

20.4 根据报错诊断

代码块

```
1 下面是完整报错：
2
3 <粘贴报错>
4
5 先不要修改文件。请说明：
6 1. 失败测试或失败命令；
7 2. 期望与实际的差异；
8 3. 根因，精确到文件和方法；
9 4. 最小修复；
10 5. 修复后要验证什么。
```

20.5 只修一个问题

代码块

```
1 只修复下面这个已确认的问题：
2 <问题描述>
```

3

- 4 要求：新增或更新一条测试；只改必要文件；不要重构其他代码；运行 `mvn test`；最后解释改动与风险。

20.6 只读审查

代码块

- 1 只读审查当前项目，不要修改文件。
- 2
- 3 重点：<边界/功能/安全/测试>
- 4 只报告高置信度问题。每条给出文件、方法、触发条件、影响、最小修复和建议测试。没有问题时明确说明。

20.7 生成文档但不编造

代码块

- 1 只根据当前代码、测试和已运行命令更新 `README.md`。
- 2
- 3 必须包括：用途、环境要求、真实可运行的命令、已实现功能、已知限制。
- 4 不要写未实现功能，不要改 `Java` 代码，不要写 `Git/GitHub` 内容。

21. 无 Git 的恢复与排错清单

21.1 如何比较修改前后

优先使用：

代码块

- ```
1 code --diff "旧文件完整路径" "新文件完整路径"
```

旧文件来自 `snapshots` 中的某一步快照；新文件来自当前 `task-tracker` 文件夹。

## 21.2 如何恢复一个文件

不要直接覆盖当前项目。先把快照文件复制到一个新位置，再比较确认。

示例：把快照中的 `TaskService.java` 复制成一个待比较文件：

```
代码块
1 copy-Item `
2 "$HOME\Documents\CodexTraining\snapshots\step-07-before-
 refactor\src\main\java\com\janel\codexlab\TaskService.java" `
3 "$HOME\Documents\CodexTraining\TaskService.before-refactor.java"
```

然后运行：

```
代码块
1 code --diff `
2 "$HOME\Documents\CodexTraining\TaskService.before-refactor.java" `
3 "$HOME\Documents\CodexTraining\task-
 tracker\src\main\java\com\janel\codexlab\TaskService.java"
```

确认后，才手动把你需要的逻辑复制回当前版本。

## 21.3 如何恢复整份项目

最安全的方法是：从快照复制出一个**新的项目文件夹**，而不是覆盖当前项目。

```
代码块
1 $root = Join-Path $HOME "Documents\CodexTraining"
2 $backup = Join-Path $root "snapshots\step-06-due-date-complete"
3 $restored = Join-Path $root "task-tracker-restored-01"
4
5 New-Item -ItemType Directory -Force -Path $restored | Out-Null
6 Get-ChildItem -Path $backup -Force | Copy-Item -Destination $restored -Recurse
 -Force
7 code $restored
```

你会得到一个新的 `task-tracker-restored-01` 文件夹。先在这个文件夹运行 `mvn test`，确认没问题后，再决定保留哪个版本。

## 21.4 Maven 常见问题

| 现象                    | 排查顺序                                                                                             |
|-----------------------|--------------------------------------------------------------------------------------------------|
| <code>mvn</code> 无法识别 | Maven 是否安装；重启 VS Code；检查 PATH                                                                    |
| 编译版本不对                | <code>java -version</code> 、 <code>javac -version</code> 、 <code>mvn -v</code><br>是否都指向 Java 17+ |

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| 单个测试失败    | 复制完整输出；先让 Codex 只读诊断                                                 |
| 所有测试都不执行  | 确认测试在 <code>src/test/java</code> ，类名或方法符合 JUnit 规则                   |
| App 主类找不到 | 用 <code>Get-ChildItem -Recurse src\main\java</code> 查看实际 package 和类名 |
| 打包成功但运行失败 | 确认你从项目根目录运行；检查 classpath 与主类全名                                       |

## 21.5 Codex 常见问题

|                  |                                         |
|------------------|-----------------------------------------|
| 现象               | 先做什么                                    |
| Codex 说项目里有不存在的类 | 让它只读重新分析，并要求附文件名和方法名证据                  |
| 它想改太多文件          | 停止，让它先给最小计划；新建快照后再继续                    |
| 它要求 Full Access  | 不选择；先问需要什么权限和为什么                        |
| 你不理解一条命令         | 不批准，要求解释该命令的输入、输出、影响范围和替代方案             |
| 它没有运行测试          | 明确要求运行 <code>mvn test</code> ，然后自己再运行一次 |
| 它修改了无关格式         | 用快照比较；要求恢复无关格式变化，只保留必要逻辑                |
| 你担心改坏了项目         | 复制快照为一个新的恢复文件夹，不要覆盖当前版本                 |

## 22. 最终验收清单

完成本教程后，确认你能够做到：

- ☐ 我能在 VS Code 安装并用 ChatGPT 账号登录 Codex。
- ☐ 我知道 API key 登录与 ChatGPT 订阅登录不是同一条计费路径。
- ☐ 我会先让 Codex 只读分析陌生代码。
- ☐ 我会用 @文件、选中代码和完整报错提供上下文。
- ☐ 我会在大改前创建本地快照，而不是依赖 Git/GitHub。
- ☐ 我会让 Codex 先计划，再决定是否实施。

- ☐ 我会让新增功能有测试，并亲自运行 `mvn test`。
- ☐ 我能提供完整失败输出，让 Codex 先诊断再修复。
- ☐ 我能区分重构与新增行为。
- ☐ 我能用 VS Code 比较快照和当前文件。
- ☐ 我写过 README.md 和 AGENTS.md。
- ☐ 我至少完成过一次 CLI 的只读分析。
- ☐ 我理解同一目录中不要让多个写入线程同时操作。
- ☐ 我创建过一个个人 Java 质量检查 Skill。
- ☐ 我知道 MCP 要先评估权限和必要性，不能因为“能连”就连接。

## 你可以记住的最终 workflow

### 代码块

- 1 创建本地快照
- 2 → 只读理解
- 3 → 明确验收标准
- 4 → 先计划
- 5 → 写/更新测试
- 6 → 最小实现
- 7 → `mvn test`
- 8 → 比较关键文件
- 9 → 创建完成快照

**最重要的一句话：Codex 可以明显加速你写代码，但不能替代你对代码、测试和文件改动负责。**

## 23. 官方参考资料

本教程中的 Codex 用法以 OpenAI 官方文档为准；产品界面和命令可能会随版本更新。

- Codex Quickstart: <https://developers.openai.com/codex/quickstart>
- Codex IDE 扩展: <https://developers.openai.com/codex/ide>
- Codex CLI: <https://developers.openai.com/codex/cli>
- CLI 参数参考: <https://developers.openai.com/codex/cli/reference>
- Agent approvals & security: <https://developers.openai.com/codex/agent-approvals-security>
- AGENTS.md: <https://developers.openai.com/codex/guides/agents-md>
- Codex App for Windows: <https://developers.openai.com/codex/app/windows>

- Automations: <https://developers.openai.com/codex/app/automations>
- Agent Skills: <https://developers.openai.com/codex/skills>